



BLOC 4 – Deep Learning

AT&T project

CDSD - Loic Valentini – Juillet 2025





Protéger les utilisateurs d'AT&T contre les SPAMs

- AT&T, leader mondial des télécommunications, sert des millions d'utilisateurs mobiles aux Etats-Unis.
- Les clients sont massivement exposés aux SMS SPAM, ce qui nuit à la confiance et à l'expérience utilisateur.
- Objectif : développer un modèle automatique capable d'identifier les SPAMs à partir du contenu du message.
- Impact stratégique : renforcer la sécurité perçue, améliorer la qualité du service, protéger la réputation d'AT&T.



Atteindre cet objectif grâce au Deep Learning

- Jeu de données : 5 572 SMS labellisés
→ SPAM : 13 % | HAM : 87 % → Fort déséquilibre des classes
- **Objectif** : Identifier l'architecture Deep Learning la plus performante pour détecter les SPAM
- **3 approches testées** :
 - TF-IDF + Réseau de Neurones : méthode classique avec vecteurs de fréquence
 - Couche d'intégration (*Embedding Layer*) + Réseau de Neurones : représentation dense du texte
 - Apprentissage par transfert (*Transfer Learning*) avec BERT : modèle pré-entraîné à large échelle
- **Évaluation des performances** :
 - Basée sur la matrice de confusion
 - Métriques clés : Exactitude (*Accuracy*), Précision (*Precision*), Rappel (*Recall*)



Une préparation rapide des données

- Nettoyage classique des données textes**

Lemmatisation, suppression des Stop_words, ponctuations, caractères spéciaux et tokenisation

- Rééquilibrage des classes :**

Sur-échantillonnage des SPAM par duplication (oversampling) pour compenser le déséquilibre initial



- Résultat final :**

Un dataset équilibré de 9 650 SMS
→ 50 % SPAM | 50 % HAM

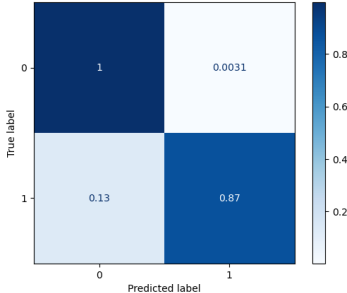
AT&T

```
label
0    4825
1    4825
Name: count, dtype: int64
```



1 : TF-IDF + Réseau de neurones

Paramètres																	
Séparation	60% train / 20% val / 20% test																
Taille du lot	64																
Réseau de neurones	<table><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr><tr><td>dense_30 (Dense)</td><td>(None, 128)</td><td>1,038,976</td></tr><tr><td>dropout_6 (Dropout)</td><td>(None, 128)</td><td>0</td></tr><tr><td>dense_31 (Dense)</td><td>(None, 64)</td><td>8,256</td></tr><tr><td>dense_32 (Dense)</td><td>(None, 1)</td><td>65</td></tr></table>		Layer (type)	Output Shape	Param #	dense_30 (Dense)	(None, 128)	1,038,976	dropout_6 (Dropout)	(None, 128)	0	dense_31 (Dense)	(None, 64)	8,256	dense_32 (Dense)	(None, 1)	65
Layer (type)	Output Shape	Param #															
dense_30 (Dense)	(None, 128)	1,038,976															
dropout_6 (Dropout)	(None, 128)	0															
dense_31 (Dense)	(None, 64)	8,256															
dense_32 (Dense)	(None, 1)	65															
Epochs	50																
Optimiseur	Adam																

Résultats										
Durée d'entraînement	≈ 1 minute									
Exactitude	0.98									
Précision	0.98									
Rappel	0.87									
Matrice de confusion	Confusion Matrix (%) <table><tr><th></th><th>Predicted label 0</th><th>Predicted label 1</th></tr><tr><th>True label 0</th><td>1</td><td>0.0031</td></tr><tr><th>True label 1</th><td>0.13</td><td>0.87</td></tr></table>		Predicted label 0	Predicted label 1	True label 0	1	0.0031	True label 1	0.13	0.87
	Predicted label 0	Predicted label 1								
True label 0	1	0.0031								
True label 1	0.13	0.87								
Poids du modèle (.keras)	12 Mo									



2 : Couche d'intégration + Réseau de neurones

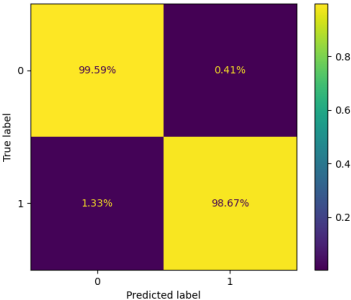
Paramètres																	
Mots conservés	Tous (8172)																
Séparation	60% train / 20% val / 20% test																
Taille du lot	64																
Réseau de neurones	<table><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr><tr><td>embedding (Embedding)</td><td>(None, 74, 64)</td><td>523,000</td></tr><tr><td>global_average_pooling1d (GlobalAveragePooling1D)</td><td>(None, 64)</td><td>0</td></tr><tr><td>dense_3 (Dense)</td><td>(None, 64)</td><td>4,160</td></tr><tr><td>dense_4 (Dense)</td><td>(None, 1)</td><td>65</td></tr></table>		Layer (type)	Output Shape	Param #	embedding (Embedding)	(None, 74, 64)	523,000	global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0	dense_3 (Dense)	(None, 64)	4,160	dense_4 (Dense)	(None, 1)	65
Layer (type)	Output Shape	Param #															
embedding (Embedding)	(None, 74, 64)	523,000															
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0															
dense_3 (Dense)	(None, 64)	4,160															
dense_4 (Dense)	(None, 1)	65															
Epochs	100																
Optimiseur	Adam																

Résultats										
Durée d'entraînement	≈ 2 minutes									
Exactitude	0.99									
Précision	0.99									
Rappel	0.91									
Matrice de confusion	Confusion Matrix (%) <table><tr><td>True label \ Predicted label</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0.0021</td></tr><tr><td>1</td><td>0.094</td><td>0.91</td></tr></table>	True label \ Predicted label	0	1	0	1	0.0021	1	0.094	0.91
True label \ Predicted label	0	1								
0	1	0.0021								
1	0.094	0.91								
Poids du modèle (.keras)	6 Mo									



3 : Apprentissage par transfert - BERT

Paramètres	
Séparation	60% train / 20% val / 20% test
Taille du lot	64
Modèle	« distilbert-base-uncased »
Epochs	5
Optimiseur	Adam, taux d'apprentissage 2e-5

Résultats										
Durée d'entraînement	> 2 heures									
Exactitude	0.99									
Précision	0.97									
Rappel	0.99									
Matrice de confusion	Confusion Matrix <table><tr><th>True label \ Predicted label</th><th>0</th><th>1</th></tr><tr><th>0</th><td>99.59%</td><td>0.41%</td></tr><tr><th>1</th><td>1.33%</td><td>98.67%</td></tr></table>	True label \ Predicted label	0	1	0	99.59%	0.41%	1	1.33%	98.67%
True label \ Predicted label	0	1								
0	99.59%	0.41%								
1	1.33%	98.67%								
Poids du modèle (.h5)	261 Mo									



Bilan et perspectives

- 3 modèles à performance élevée mais néanmoins variable (Bert étant significativement meilleur), détectant une grande majorité de spams
- Des différences d'approche qui affectent les durées d'entraînement et la taille des modèles
- Le choix du modèle dépendra du contexte opérationnel :
 - > Déploiement local ou cloud ? Critères : poids du modèle et latence
 - > Réentraînement nécessaire ? Critère : complexité de pipeline MLOps
 - > Temps de réponse critique ? Critère : légèreté
 - > Performance optimale ? Critère : metrics



Merci !

